Neural Networks for NLP

Tim Book

GENERAL ASSEMBLY

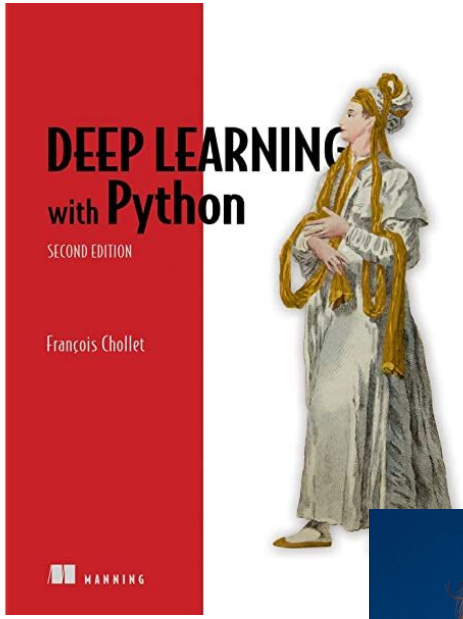
Agenda

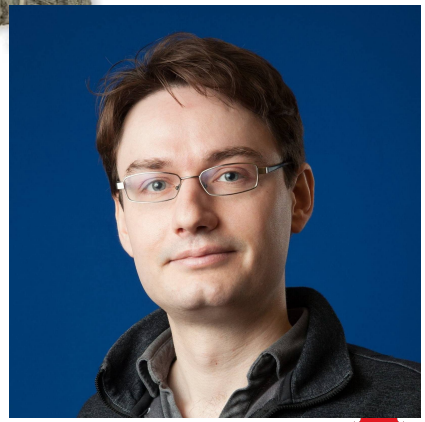
1. Word vectors: An alternative to the CountVectorizer!
2. Formatting documents as word vectors for modeling
3. Using RNNs for our model
4. Python!

 © 2022 General Assembly

Citation

To give credit where credit is due, many of the examples and images used in this lesson are from **Deep Learning with Python** by **François Chollet**, the author of the Keras library!





Document Representation



When we did classification NLP models a few weeks ago, we represented entire documents as a row of our **X** matrix. There were two methods for this. What were they?

 © 2022 General Assembly

Document Representation: Count Vectorizing

The most popular way (so far) to represent a document was by **count vectorizing** it. This involved creating one column per term in our vocabulary. For example:

 © 2022 General Assembly

Document Representation: Count Vectorizing

Document 1: “This restaurant was bad”
Document 2: “The restaurant had mice”

this	restaurant	was	bad	the	had	mice
1	1	1	1	0	0	0
0	1	0	0	1	1	1

 © 2022 General Assembly

Document Representation: Count Vectorizing

Another way of thinking about this is that each document is made up of **word vectors**. And the document is the sum of them. For example:

 © 2022 General Assembly

Document Representation: Count Vectorizing

the restaurant had mice = the + restaurant + had + mice

$$\begin{bmatrix}0\\1\\0\\0\\1\\1\\1\end{bmatrix} \quad \begin{bmatrix}0\\0\\0\\0\\1\\0\\0\end{bmatrix} \quad \begin{bmatrix}0\\1\\0\\0\\0\\0\\0\end{bmatrix} \quad \begin{bmatrix}0\\0\\0\\0\\0\\1\\0\end{bmatrix} \quad \begin{bmatrix}0\\0\\0\\0\\0\\0\\1\end{bmatrix}$$

 © 2022 General Assembly

What's wrong with this? Part I

1. The whole point of neural networks is that we allow our model to learn feature engineering from exposure to the data alone! We're making an explicit choice to count vectorize.
2. Count vectorization assumes our word vectors are **independent** (they dot-product to 0). This is often not true! For example "movie" and "film" are almost always interchangeable words but would be represented as totally independent!

 © 2022 General Assembly

What's wrong with this? Part II

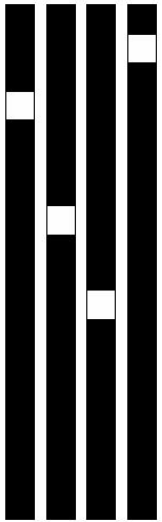
3. These word vectors are **large** and **sparse**! Is there a more efficient way to represent them? For example, in our previous example, our word vectors were in $\mathbb{R}^7$. Could they be equally represented in $\mathbb{R}^3$ or $\mathbb{R}^4$?

4. When we did this, we used **bag-of-words** modeling. Each observation was an entire document.

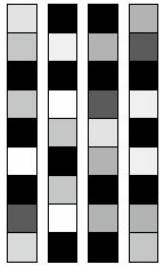
Enter: Word Embeddings!

Instead of these **high-dimensional**, **sparse** representations, we can use **word embeddings** to produce **low-dimensional**, **dense** representations ***that can be learned from the data!***

For example, typical vocabularies are about 20K words. It's common to see Word embeddings that are fewer than 1K-dimensional.



One-hot word vectors:
- Sparse
- High-dimensional
- Hardcoded



Word embeddings:
- Dense
- Lower-dimensional
- Learned from data

 © 2022 General Assembly

Creating your own Embeddings

While these words *can* be learned from your data, generally you need a lot of data for this, and a compelling use case to do so.

We won't be doing so, but it's not hard. It's as simple as passing your data through `keras.layers.Embedding()` before the RNN layers.

Instead, we'll opt to use **pre-trained embeddings!** Even if you did have enough data to train yourself, this is still usually preferred.

GloVe: Global Vectors for Word Representation

The most commonly used pre-trained word embeddings dataset is the **GloVe** data.

There are several different versions of GloVe, each trained on different sources. The one we'll be using today is trained on:

- The entirety of Wikipedia (as of 2014)
- The Gigaword 5 corpus (many years of news articles from several major American and international news outlets through 2010).

The combined corpus has 6B tokens consisting of a 400K token vocab.

You can read more about GloVe here.

GloVe's "Nearest Neighbors"

1. Nearest neighbors

The Euclidean distance (or cosine similarity) between two word vectors provides an effective method for measuring the linguistic or semantic similarity of the corresponding words. Sometimes, the nearest neighbors according to this metric reveal rare but relevant words that lie outside an average human's vocabulary. For example, here are the closest words to the target word *frog*:

0. *frog*
1. frogs
2. toad
3. litoria
4. leptodactylidae
5. rana
6. lizard
7. eleutherodactylus



3. litoria



4. leptodactylidae



5. rana

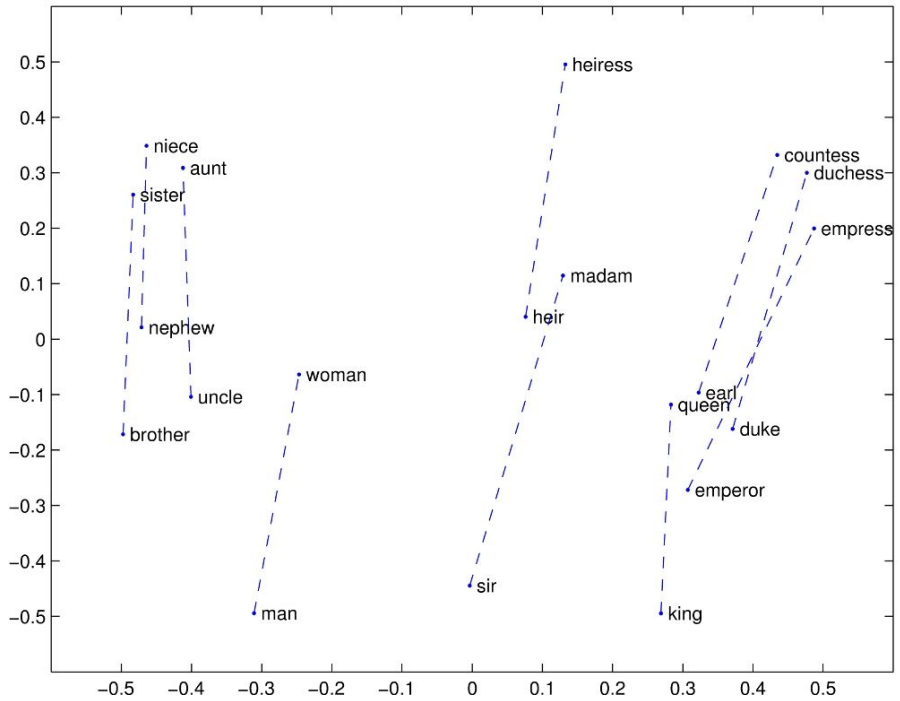


7. eleutherodactylus

 © 2022 General Assembly

GloVe's "Linear Substructures"

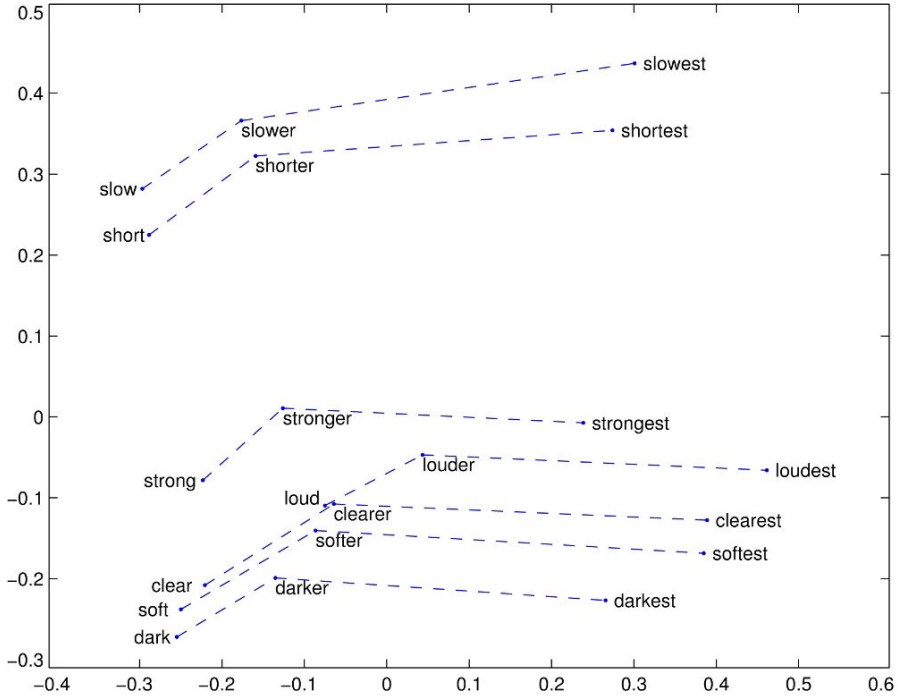
The idea is that the word vectors have an additive vector structure that we might interpret as an "analogy".



https://nlp.stanford.edu/projects/glove/images/man_woman.jpg

GloVe's "Linear Substructures"

The idea is that the word vectors have an additive vector structure that we might interpret as an "analogy".



https://nlp.stanford.edu/projects/glove/images/comparative_superlative.jpg

 © 2022 General Assembly

Word Embeddings in Neural Networks

Word Embeddings in Neural Networks

OLD:

One “observation” for an NLP model is the sum of count vectorized word vectors. (ie, a 1-dim vector)

NEW:

One “observation” for an NLP model is **a sequence of word vectors.** (ie, a 2-dim matrix)

 © 2022 General Assembly

Data Setup

We must first choose two numbers:

1. **Word embedding dimension (*D*)** - GloVe gives us a few options. We'll be using the 100-dim vectors today, but options range from 50-dim to 300-dim.
2. **Maximum sequence length (*L*)** - All sequences must be the same length for RNNs. Documents below the chosen length will be 0-padded, and documents above this length will be truncated.

Each document is therefore a matrix of dimension (D, L). If we have N samples, our full corpus is dimension (N, D, L). That's big!

 © 2022 General Assembly

Example

$D = 100$

$L = 8$

Document = “the cat sat on the mat”

the	cat	sat	on	the	mat	[blank]	[blank]
-0.038	0.231	-0.153	-0.219	-0.038	-0.848	0	0
-0.245	0.283	0.485	-0.427	-0.245	0.440	0	0
0.728	0.632	0.347	0.520	0.728	-0.121	0	0
-0.400	-0.594	0.179	0.004	-0.400	-1.193	0	0
…	…	…	…	…	…	…	…

Some Keras Notes

Note #1: New Notation

Up until now, we've been using the **sequential** API for Keras. Reminder, that looks like this:

```
model = Sequential()
model.add(Dense(...))
model.add(Dropout(...))
model.add(Dense(...)
```

Note #1: New Notation

Just to mix things up, we'll be using the **functional** API for Keras. Everything's still there, but it looks a little different:

```
input = Input(...)
x = Dense(...)(input)
x = Dropout(...)(x)
outputs = Dense(...)(x)
model = Model(inputs, outputs)
```

Note #2: Bidirectional RNNs

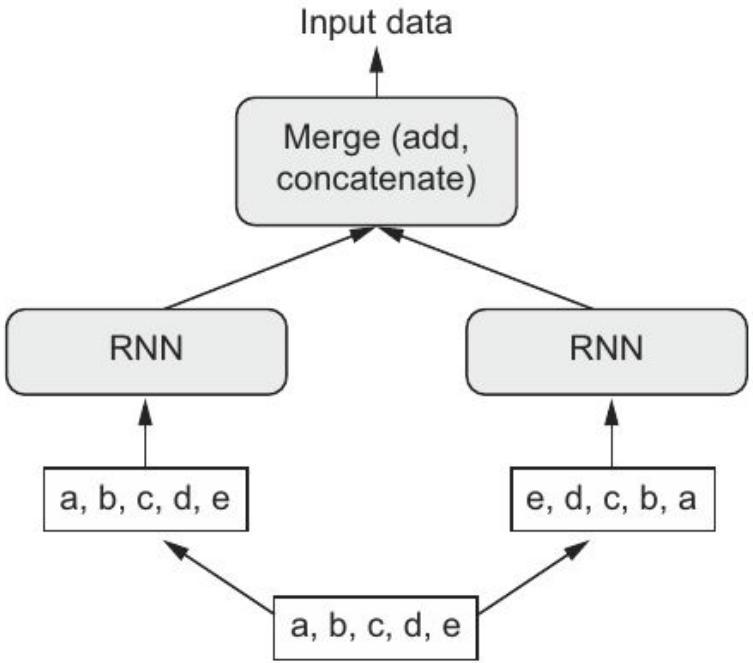


Figure 10.14 How a bidirectional RNN layer works

Deep Learning with Python, 2nd Edition, François Chollet

Let's try it out!



 © 2022 General Assembly